HOA SEN UNIVERSITY

www.hoasen.edu.vn

Laravel 2

ROUTING & MIDDLEWARE

Contents

- Routes
  - Basic Routing
  - The Default Route Files
  - Available Router Methods
  - View Routes & Redirect Routes
  - Route Parameters
  - Global Constraints
  - Named Routes
  - Route Groups
- Middleware
  - Defining Middleware
  - Registering Middleware
  - Assigning Middleware To Routes
  - Middleware Groups
  - Middleware Parameters
- CSRF Protection
- Redirects

Basic Routing

- The most basic Laravel routes accept a URI and a Closure, providing a very simple and expressive method of defining routes:

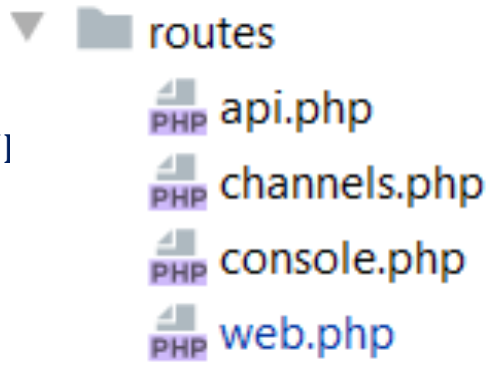
```
Route::get('foo', function () {
    return 'Hello World';
});
```

What's a Closure?

Closures are PHP's version of anonymous functions. A closure is a function that you can pass around as an object, assign to a variable, pass as a parameter to other functions and methods, or even serialize.

The Default Route Files

- All Laravel routes are defined in your route files, which are located in the routes directory
  - routes/web.php : be accessed by entering the defined route's Ul



7.x

```
Route::get('/user', 'UserController@index');
```

```
Route::get('/user', [UserController::class, 'index']);
```

8.x

http://your-app.test/user

  - routes/api.php : are stateless and are assigned the api middleware group
    - Routes defined in the routes/api.php file are nested within a route group by the RouteServiceProvider. Within this group, the /api URI prefix is automatically applied so you do not need to manually apply it to every route in the file.

Available Router Methods

```
Route::get($uri, $callback);
Route::post($uri, $callback);
Route::put($uri, $callback);
Route::patch($uri, $callback);
Route::delete($uri, $callback);
Route::options($uri, $callback);
```

```
Route::match(['get', 'post'], '/', function () {
    //
});

Route::any('/', function () {
    //
});
```

CSRF Protection

- CSRF : Cross-site Request Forgery
- Any HTML forms pointing to POST, PUT, PATCH, or DELETE routes that are defined in the web routes file should include a CSRF token field. Otherwise, the request will be rejected.

```
<form method="POST" action="/profile">
    @csrf
    ...
</form>
```

View Routes & Redirect Routes

- This method provides a convenient shortcut so that you do not have to define a full route or controller for performing a simple redirect

```
Route::redirect('/here', '/there');
```

- By default, Route::redirect returns a 302 status code. You may customize the status code using the optional third parameter:

```
Route::redirect('/here', '/there', 301);
```

```
Route::permanentRedirect('/here', '/there');
```

```
Route::view('/welcome', 'welcome');

Route::view('/welcome', 'welcome', ['name' => 'Taylor']);
```

Route Parameters

- Required Parameters

```
Route::get('posts/{post}/comments/{comment}', function ($postId, $commentId) {
    //
});
```

- Optional Parameters

```
Route::get('user/{name?}', function ($name = null) {
    return $name;
});

Route::get('user/{name?}', function ($name = 'John') {
    return $name;
});
```

Regular Expression Constraints

- You may constrain the format of your route parameters using the **where** method on a route instance.
- The **where** method accepts the name of the parameter and a regular expression defining how the parameter should be constrained:

```
Route::get('user/{name}', function ($name) {
    //
})->where('name', '[A-Za-z]+');

Route::get('user/{id}', function ($id) {
    //
})->where('id', '[0-9]+');

Route::get('user/{id}/{name}', function ($id, $name) {
    //
})->where(['id' => '[0-9]+', 'name' => '[a-z]+']);
```

Global Constraints

app/providers/RouteServiceProvider.php

```php
/**
 * Define your route model bindings, pattern filters, etc.
 *
 * @return void
 */
public function boot()
{
    Route::pattern('id', '[0-9]+');

    parent::boot();
}
```

```php
Route::get('user/{id}', function ($id) {
    // Only executed if {id} is numeric...
});
```

Named Routes

```
Route::get('user/profile', function () {
    //
})->name('profile');
```

```
Route::get('user/profile', 'UserProfileController@show')->name('profile');
```

```
// Generating URLs...
$url = route('profile');

// Generating Redirects...
return redirect()->route('profile');
```

- Route names should always be unique.

Named Routes

```
Route::get('user/{id}/profile', function ($id) {
    //
})->name('profile');

$url = route('profile', ['id' => 1]);
```

- If you pass additional parameters in the array, those key / value pairs will automatically be added to the generated URL's query string:

```
Route::get('user/{id}/profile', function ($id) {
    //
})->name('profile');

$url = route('profile', ['id' => 1, 'photos' => 'yes']);

// /user/1/profile?photos=yes
```

Route Groups

- Routes Groups are beneficial when you want to apply the attributes to all the routes. If you use route groups, you do not have to apply the attributes individually to each route; this avoids duplication.

```
Route::group(['prefix' => 'public'], function()
{
    Route::get( uri: '/first',function()
    {
        echo "first route";
    });
    Route::get( uri: '/second',function()
    {
        echo "second route";
    });
    Route::get( uri: '/third',function()
    {
        echo "third route";
    });
});
```

http://127.0.0.1:8000/public/first
http://127.0.0.1:8000/public/second
http://127.0.0.1:8000/public/third

Accessing The Current Route

- You may use the current, currentRouteName, and currentRouteAction methods on the Route facade to access information about the route handling the incoming request:

```
$route = Route::current();

$name = Route::currentRouteName();

$action = Route::currentRouteAction();
```

MIDDLEWARE

Introduction

- Middleware provide a convenient mechanism for filtering HTTP requests entering your application
- There are several middleware included in the Laravel framework, including middleware for authentication and CSRF protection. All of these middleware are located in the app/Http/Middleware directory.

Defining Middleware

```
php artisan make:middleware CheckAge
```

- This command will place a new CheckAge class within your app/Http/Middleware directory.

```
class CheckAge
{
    public function handle($request, Closure $next)
    {
        if ($request->age > 200) {
            return redirect('/checkfail');
        }
        return $next($request);
    }
}
```

Registering Middleware

app.php

tailwindapp > bootstrap > app.php

```php
<?php

use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        //bo sung middleware cho web routes
        $middleware->web(append: [
            \App\Http\Middleware\CheckAge::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();

```

Assigning Middleware To Routes

```
//test middleware Age
//routes/web.php
use App\Http\Middleware\CheckAge;

Route::get('admin/profile', function () {
    echo "welcome!";
})->middleware(CheckAge::class);
```

```
Route::middleware([CheckAge::class])->group(function () {
    Route::get('/', function () {
        //
    });
    Route::get('admin/profile', function () {
        //
    })->withoutMiddleware([CheckAge::class]);
});
```

Middleware Groups

```
Route::get('/', function () {
    //
})->middleware('web');

Route::group(['middleware' => ['web']], function () {
    //
});

Route::middleware(['web', 'subscribed'])->group(function () {
    //
});
```

NOTE:

The web middleware group is automatically applied to your routes/web.php file by the RouteServiceProvider.

Middleware Parameters

App/Http/Middleware/CheckRole.php

```
class CheckRole
{
    /**
     * Handle an incoming request.     *
     * @param  \Illuminate\Http\Request  $request
     * @param  \Closure  $next
     * @return mixed
     */
    public function handle($request, Closure $next, $role)
    {

        if(!$request->user()){
            return redirect("login");
        }
        else if (! $request->user()->hasRole($role))
            return redirect("login");
        }
        return $next($request);
    }
}
```

bootstrap/app.php

```
protected $routeMiddleware = [
    ....
    'role' =>\App\Http\Middleware\CheckRole::class,
];
```

routes/web.php

```
Route::put('post/{id}', function ($id) {
//
 })->middleware('role:editor');
```

CSRF Protection

- CSRF - Cross-Site Request Forgery attacks
- Cross-site request forgeries are a type of malicious exploit whereby unauthorized commands are performed on behalf of an authenticated user.
- Laravel automatically generates a CSRF "token" for each active user session managed by the application. This token is used to verify that the authenticated user is the one actually making the requests to the application.
- You should include a hidden CSRF token field in the form so that the CSRF protection middleware can validate the request

```
<form method="POST" action="/profile">
    @csrf
    ...
</form>
```

- The VerifyCsrfToken middleware, which is included in the web middleware group, will automatically verify that the token in the request input matches the token stored in the session.

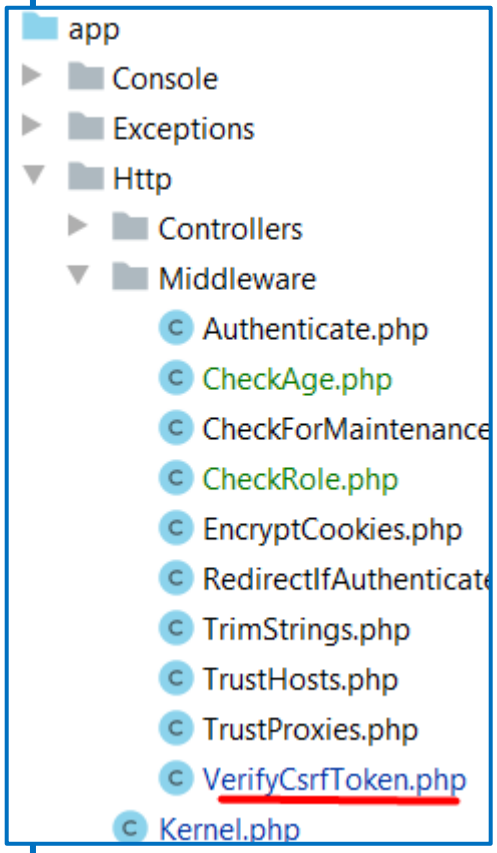
Excluding URIs From CSRF Protection

```php
<?php

namespace App\Http\Middleware;

use Illuminate\Foundation\Http\Middleware\VerifyCsrfToken as
Middleware;

class VerifyCsrfToken extends Middleware
{
    /**
     * The URIs that should be excluded from CSRF verification.
     *
     * @var array
     */
    protected $except = [
            'http://example.com/foo/bar',
            'http://example.com/foo/*',
    ];
}
```



The CSRF middleware is automatically disabled when running tests.

X-CSRF-TOKEN

- To checking for the CSRF token as a POST parameter, the VerifyCsrfToken middleware will also check for the X-CSRF-TOKEN request header.

```
<!-- CSRF Token -->
<meta name="csrf-token" content="{{ csrf_token() }}">
```

- Once you have created the meta tag, you can instruct a library like jQuery to automatically add the token to all request headers. This provides simple, convenient CSRF protection for your AJAX based applications:

```
$.ajaxSetup({ headers: { 'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content') } });
```

- Laravel stores the current CSRF token in an encrypted XSRF-TOKEN cookie that is included with each response generated by the framework. You can use the cookie value to set the X-XSRF-TOKEN request header.

Redirects (1)

- There are two common ways to generate a redirect:
  - use the redirect() global helper
  - use Redirect facade

```
// Using the global helper to generate a redirect response
Route::get('redirect-with-helper', function () {
    return redirect()->to('login');
});
// Using the global helper shortcut
Route::get('redirect-with-helper-shortcut', function () {
    return redirect('login');
});
// Using the facade to generate a redirect response
Route::get('redirect-with-facade', function () {
    return Redirect::to('login');
});
// Using the Route::redirect shortcut in Laravel 5.5+
Route::redirect('redirect-by-route', 'login');
```

Redirects (2)

- redirect()->to()
  - The method signature for the to() method for redirects looks like this:

```
function to($to = null, $status = 302, $headers = [], $secure = null)
```

```
Route::get('redirect', function () {
    return redirect()->to('home');
    // Or same, using the shortcut:
    return redirect('home');
});
```

- redirect()->route()
  - The route() method is the same as the to() method, but rather than pointing to a particular path, it points to a particular route name

```
function route($to = null, $parameters = [], $status = 302, $headers = [])
```

```
Route::get('redirect', function () {
    return redirect()->route('conferences.index');
});
Route::get('redirect', function () {
    return redirect()->route('conferences.show', ['conference' => 99]);
});
```

Redirects (3)

- redirect()->back()
  - redirects the user to whatever page they came from
- Other Redirect Methods
  - home() : redirects to a route named home
  - refresh(): redirects to the same page the user is currently on
  - away(): redirecting to an external URL without the default URL validation
  - secure() : like to() with the secure parameter set to "true“
  - action() : Allows you to link to a controller and method in one of two ways:
    - as a string: redirect()->action('MyController@myMethod’)
    - or as a tuple: redirect()->action([MyController::class, 'myMethod’])

Redirects (4)

- Redirecting With Flashed Session Data

```
Route::get('redirect-with-key-value', function () {
    return redirect('dashboard')
        ->with('error', true);
});
Route::post('user/profile', function () {
        // Update the user's profile...
        return redirect('dashboard')
        ->with('status', 'Profile updated!');
});
```

```
@if (session('status'))
        <div class="alert alert-success"> {{ session('status') }} </div>
@endif
```

Q&A